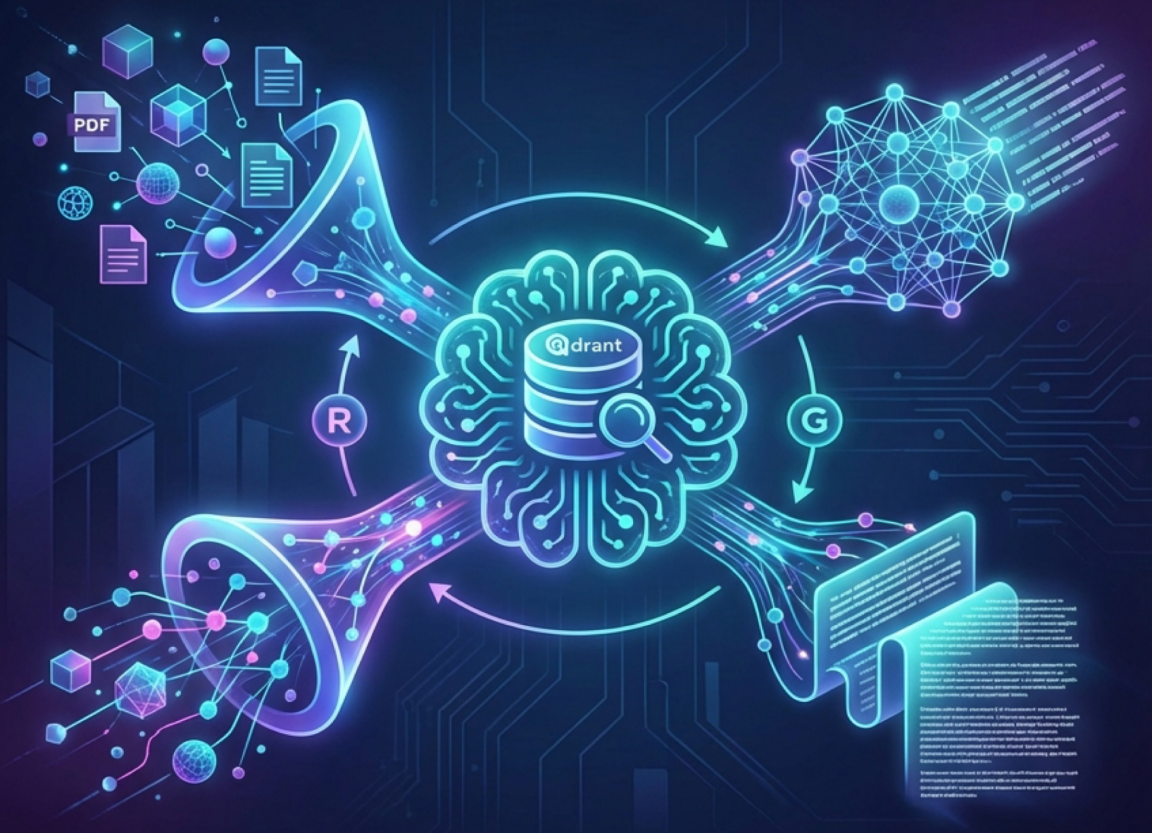
Blink
Fundamentos de RAG



Integração Inteligente de Modelos de Linguagem com Dados Reais

Daniel Romero

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

Blink Fundamentos de RAG

Alberto Souza

2026-02-23

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

Contents

O que tem neste blink para mim? i

RAG combina o melhor de dois mundos: recuperação de informação e geração de texto 1

O fluxo completo do RAG começa pela ingestão inteligente dos dados 3

Embeddings transformam texto em vetores numéricos que preservam significado 5

A cosine similarity mede o ângulo entre vetores para encontrar textos parecidos 7

Um sistema RAG básico pode ser construído do zero com poucas linhas de Python 9

Vector databases como Qdrant são indispensáveis para escalar o RAG além da memória 11

Docling transforma documentos não estruturados em texto limpo para alimentar o RAG 13

A estratégia de chunking define a qualidade do retrieval e não pode ser arbitrária 15

Metadados enriquecem os chunks e são o diferencial entre um RAG medíocre e um excelente 17

O pipeline completo de ingestão une extração, chunking, embeddings e vector database em um fluxo coeso 19

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

Resumo Final 21
Sobre este resumo 22

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

O que tem neste blink para mim?

Você provavelmente já ouviu falar de modelos de linguagem como ChatGPT, Claude e Llama. Eles impressionam pela capacidade de gerar texto coerente e contextualmente adequado. Mas existe um problema fundamental: esses modelos dependem exclusivamente dos dados com os quais foram treinados. Quando precisam responder perguntas sobre documentos internos da sua empresa, artigos recentes ou qualquer informação que não fez parte do treinamento, eles inventam respostas – um fenômeno conhecido como alucinação.

E se houvesse uma forma de dar a esses modelos acesso a uma base de conhecimento real, fazendo com que as respostas fossem fundamentadas em documentos concretos? Essa é exatamente a proposta da arquitetura RAG – Retrieval-Augmented Generation – que combina o melhor da recuperação de informações com a geração de texto.

Neste blink, baseado no curso "Fundamentos de RAG" de Daniel Romero, você vai descobrir:

- como a arquitetura RAG integra retrieval e generation para eliminar alucinações;
- o que são embeddings e como eles capturam o significado semântico dos textos;
- como a cosine similarity permite medir a distância entre conceitos;
- por que vector databases como Qdrant são essenciais para escalar sistemas RAG;
- como extrair dados de PDFs, HTMLs e documentos complexos com Docling;

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

- por que a estratégia de chunking faz toda a diferença na qualidade das respostas;
- como enriquecer chunks com metadados para melhorar a precisão do retrieval;
- como montar um pipeline completo de ingestão de dados com FastEmbed;
- a importância de alinhar o tokenizer com o modelo de embeddings;
- e como construir um sistema RAG funcional do zero, em Python.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

RAG combina o melhor de dois mundos: recuperação de informação e geração de texto

Modelos de linguagem sozinhos enfrentam uma limitação crítica: eles dependem exclusivamente dos dados de treinamento para gerar respostas. Quando você faz uma pergunta sobre um documento que o modelo nunca viu, ele não tem como saber a resposta correta – e, na maioria das vezes, vai inventar algo que parece convincente mas não é verdadeiro. Por outro lado, sistemas de Information Retrieval conseguem buscar documentos relevantes, mas não são bons em gerar texto coerente e contextualmente adequado.

A arquitetura RAG – Retrieval-Augmented Generation – resolve esse dilema ao combinar essas duas abordagens. Em essência, o RAG possui três componentes principais: o Input Query (a pergunta do usuário), o Retriever (que busca documentos relevantes em um grande corpus) e o Generator (o LLM que gera a resposta com base nos documentos recuperados). O Retriever utiliza técnicas como Vector Similarity Search ou busca baseada em keywords para identificar os trechos mais relevantes. Esses trechos formam o contexto que será passado ao LLM, que então gera uma resposta fundamentada em informações reais.

> *Ao combinar essas duas abordagens, a arquitetura RAG nos dá o melhor dos dois mundos. Você vai aprender a melhorar a precisão das respostas dos modelos e aproveitar melhor os documentos.*

O RAG não substitui o LLM – ele o complementa. A ideia central é que a geração de texto seja guiada pelo conteúdo real dos documentos, evitando que o modelo invente informações. Pense no RAG como dar ao modelo um "livro de consulta" antes de

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

responder.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

O fluxo completo do RAG começa pela ingestão inteligente dos dados

Antes de qualquer busca por similaridade ou geração de resposta, o RAG precisa de uma etapa fundamental: a ingestão dos dados. Esse processo transforma documentos brutos – PDFs, páginas HTML, planilhas, vídeos, imagens – em representações vetoriais que podem ser consultadas de forma eficiente. Essa etapa é tão importante quanto a geração em si, porque a qualidade das respostas depende diretamente da qualidade da base de conhecimento.

O fluxo de ingestão começa com o Text Loader, que carrega os documentos de diversas fontes. Em seguida, entra o Splitter, que divide o conteúdo em partes menores chamadas chunks. Cada chunk pode ser enriquecido com metadados – título do documento, página, data de criação, autor, seção. Depois, cada chunk é transformado em um embedding, um vetor numérico que captura o significado semântico do texto. Esses embeddings são armazenados em um Vector Database, uma base de dados especializada que permite buscas rápidas por similaridade. Quando o usuário faz uma pergunta, essa pergunta também é convertida em embedding e comparada com os vetores armazenados, retornando os chunks mais relevantes.

> *O processo completo é o que garante que a geração de texto seja guiada pelo conteúdo real, evitando que o modelo invente informações. Com isso, temos uma visão completa de como o RAG integra a recuperação de informações e geração de texto para entregar respostas confiáveis e contextualizadas.*

Pense no fluxo de ingestão como a construção de uma biblioteca

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

organizada. Não basta ter os livros – você precisa catalogá-los, indexá-los e organizá-los para que qualquer pessoa encontre rapidamente o que precisa. Sem essa organização, o LLM receberia informação desordenada e as respostas perderiam qualidade.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

Embeddings transformam texto em vetores numéricos que preservam significado

Para que um computador consiga comparar a similaridade entre textos, ele precisa de uma representação matemática desses textos. É exatamente isso que os embeddings fazem: transformam palavras, frases ou parágrafos inteiros em vetores de números – sequências de valores de ponto flutuante que capturam o significado semântico do conteúdo. Dois textos com significados parecidos terão vetores próximos no espaço multidimensional; textos sem relação terão vetores distantes.

Os modelos de embeddings, como o all-MiniLM-L6-v2 usado no curso, são treinados olhando para milhões de pares de textos. Alguns pares são positivos (duas frases que dizem a mesma coisa, ou uma pergunta com a resposta certa) e outros são negativos (frases sem relação). O treino funciona dizendo ao modelo: "esses dois aqui têm que ficar perto, e esses outros têm que ficar distantes". Depois de repetir isso milhares de vezes, o modelo aprende a transformar qualquer texto em um vetor que preserva o sentido. No caso do all-MiniLM-L6-v2, cada texto é transformado em um vetor de 384 dimensões – ou seja, 384 números de ponto flutuante de 32 bits que representam o significado daquele texto.

> *Esses modelos de embeddings são treinados olhando para milhões de pares de textos. Alguns pares são positivos, como duas frases que querem dizer a mesma coisa, ou uma pergunta com a resposta certa. O treino funciona basicamente dizendo para o modelo: esses dois aqui têm que ficar perto, e esses outros têm que ficar distante.*

A escolha do modelo de embeddings importa muito. O all-MiniLM-L6-v2 é rápido e leve, mas funciona melhor para textos em inglês.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

Para conteúdo em português, considere modelos multilinguais. Além disso, lembre-se de que cada modelo tem um limite máximo de tokens de entrada (512 para o MiniLM) – textos maiores que esse limite serão cortados.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

A cosine similarity mede o ângulo entre vetores para encontrar textos parecidos

Depois de transformar textos em vetores, precisamos de uma forma matemática de medir o quanto dois vetores são parecidos. A cosine similarity resolve isso calculando o cosseno do ângulo entre dois vetores no espaço multidimensional. A fórmula é direta: o produto escalar dos dois vetores dividido pelo produto das suas normas euclidianas. O resultado varia de -1 a 1: quando é 1, os vetores apontam na mesma direção (textos muito similares); quando é 0, são perpendiculares (sem relação); e -1 indica o oposto completo – embora isso raramente aconteça com textos na prática.

O que torna a cosine similarity especialmente útil para embeddings é que ela mede apenas a direção dos vetores, ignorando o comprimento. Ao dividir pelo produto das normas, normalizamos o cálculo para focar exclusivamente na orientação dos vetores no espaço. Em Python, a implementação cabe em uma única linha usando NumPy: o produto escalar com `np.dot(a, b)` dividido pela multiplicação das normas `np.linalg.norm(a) * np.linalg.norm(b)`. Mesmo que estejamos medindo o ângulo entre vetores de 384 dimensões cada, o NumPy faz toda a matemática pesada.

> *A similaridade do cosseno mede o ângulo entre as setas, não o comprimento delas. Por isso que normalizamos, ou dividimos pelas normas, pra focar só na direção e ignorar o tamanho das setas.*

Entender a cosine similarity é fundamental para depurar problemas em sistemas RAG. Se os resultados da busca não estão relevantes, o problema pode estar nos embeddings (modelo inade-

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

quado) ou na métrica de distância. A cosine similarity é a métrica mais comum, mas existem alternativas como distância euclidiana e produto escalar. Cada uma tem vantagens dependendo do cenário.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

Um sistema RAG básico pode ser construído do zero com poucas linhas de Python

A melhor forma de entender como o RAG funciona é construir um do zero, sem frameworks ou abstrações. Com apenas três bibliotecas – NumPy para cálculos, Sentence Transformers para embeddings e Groq para o LLM – é possível criar um sistema funcional. O processo se divide em três funções principais: retrieve (busca), generate_answer (geração) e a função rag que orquestra tudo.

A função retrieve converte a pergunta do usuário em um embedding, calcula a cosine similarity com todos os documentos armazenados e retorna os top-k mais similares, ordenados por score. A função generate_answer monta o prompt combinando o system prompt (instruções), o contexto recuperado (os documentos relevantes) e a pergunta original do usuário, enviando tudo para o LLM com temperatura zero para respostas determinísticas. Na função principal rag, primeiro recuperamos os documentos, depois geramos a resposta. Esse fluxo simples – buscar contexto e alimentar o LLM – é a essência de qualquer sistema RAG, independente da complexidade.

> *Claro que em um projeto real isso seria bem diferente, teríamos um banco de dados vetorial, estratégia de extração de conteúdo, chunking, várias práticas mais sofisticadas, adicionaríamos também filtros e metadados e muito mais, mas essa é a base, e agora você entende como o RAG funciona na prática.*

A temperatura zero no LLM é crucial em sistemas RAG. Ao eliminar a aleatoriedade na geração, você garante que as respostas sejam determinísticas e focadas exclusivamente no contexto fornecido.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

Isso reduz drasticamente o risco de alucinações, que é exatamente o problema que o RAG se propõe a resolver.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

Vector databases como Qdrant são indispensáveis para escalar o RAG além da memória

Guardar vetores na memória com NumPy funciona bem para 11 documentos, mas não para 10 mil, 100 mil ou 50 milhões. Para cada busca, o sistema básico calcula a similaridade com todos os documentos – complexidade O(n). Com um milhão de documentos, você vai esperar um bom tempo pela resposta. Os vector databases resolvem esse problema usando algoritmos de busca aproximada como o HNSW (Hierarchical Navigable Small World), que criam índices inteligentes permitindo encontrar os vizinhos mais próximos muito mais rapidamente, sem comparar com todos os vetores.

Qdrant é um vector database escrito em Rust, rápido e com API amigável. No Qdrant, os dados são organizados em collections (equivalentes a tabelas em bancos tradicionais), e cada documento é representado por um Point – uma estrutura que combina um ID único, o vector embedding e um payload com metadados. A migração do RAG básico para o Qdrant simplifica o código drasticamente: em vez de implementar cosine similarity na mão, basta chamar `query_points` passando a query, o limite de resultados e `with_payload=True`. Internamente, o Qdrant faz toda a matemática e retorna os resultados ordenados por similaridade.

> *Quando você tem apenas 11 documentos, guardar tudo na memória funciona tranquilo. Mas se você tiver 10 mil documentos, 100 mil documentos, 50 milhões de documentos, a memória não vai aguentar, e mesmo que aguente, as buscas vão ficar lentas demais.*

Além da velocidade, os vector databases oferecem persistência (vetores salvos em disco), escalabilidade (milhões de vetores), fil-

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

tros por metadados e facilidade de atualização. O Qdrant pode ser usado tanto em memória (para testes) quanto com persistência em disco (para produção), bastando trocar o parâmetro de `location=":memory:"` para `path="db/data"`.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

Docling transforma documentos não estruturados em texto limpo para alimentar o RAG

Existe uma estimativa de que 90% dos dados das organizações são dados não estruturados – PDFs, documentos Word, páginas HTML, planilhas. Extrair texto desses formatos de maneira confiável é um desafio técnico real: PDFs têm imagens com texto embutido, tabelas complexas, layouts variados. A maioria das ferramentas tem limitações sérias nessa tarefa. O Docling, uma biblioteca Python de código aberto, resolve esse problema de forma robusta – e já foi usado pelo instrutor em um pipeline com mais de 50 milhões de documentos.

O Docling funciona com um Document Converter que extrai toda a estrutura interna de um documento – títulos, parágrafos, tabelas, imagens – e permite exportar para Markdown. Essa conversão para Markdown é especialmente valiosa porque os LLMs são treinados extensivamente com esse formato, compreendendo melhor tabelas e estruturas formatadas do que texto solto. O Docling aceita tanto arquivos locais quanto URLs, e pode processar PDFs, HTMLs e outros formatos de maneira idêntica. Para imagens embutidas em PDFs, basta configurar o pipeline com `generate_picture_imag` e definir a escala de resolução desejada.

> *Independente de qual vector database, ou embedding model, ou LLM um projeto esteja usando, essa etapa que veremos é o que vai te ajudar a construir uma base de conhecimento para sua aplicação.*

O formato Markdown é o "idioma preferido" dos LLMs para dados estruturados. Quando você exporta uma tabela como Markdown em vez de texto puro, o modelo compreende a relação entre linhas e colunas muito melhor. Por isso, o Docling exportando

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

para Markdown não é apenas uma conveniência – é uma decisão técnica que melhora a qualidade do RAG.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

A estratégia de chunking define a qualidade do retrieval e não pode ser arbitrária

Dividir documentos em chunks não é tão simples quanto cortar o texto a cada X palavras ou caracteres. Uma divisão arbitrária pode partir um parágrafo ao meio, separar uma pergunta da sua resposta ou isolar um título do seu conteúdo – destruindo o significado semântico que o embedding deveria capturar. O Docling oferece duas estratégias complementares: o Hierarchical Chunker, que respeita a estrutura lógica do documento (títulos, subtítulos, parágrafos completos), e o Hybrid Chunker, que combina essa análise estrutural com o controle do número máximo de tokens.

O Hybrid Chunker é a estratégia mais sofisticada. Ele recebe três parâmetros: um tokenizer (alinhado com o modelo de embeddings), o limite máximo de tokens por chunk e a opção `merge_peers` para mesclar elementos adjacentes do mesmo nível hierárquico. A combinação é poderosa: chunks que fazem sentido semanticamente e que cabem dentro do limite de entrada do modelo de embeddings. No caso do all-MiniLM-L6-v2, o limite máximo é 512 tokens de entrada, então configurar chunks de até 300 tokens garante uma margem de segurança. Se um chunk exceder o limite, o modelo simplesmente corta a informação – e você perde dados.

> *Todo modelo de embedding tem um input máximo que podemos usar. Na sua estratégia de chunking, você precisa manter seus chunks abaixo desse número específico do embedding model que estiver em uso.*

A regra de ouro do chunking: não conte por palavras ou car-

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

acteres – conte por tokens. Cada modelo tem seu próprio vocabulário e forma de tokenizar o texto. Um token pode ser uma palavra inteira, um pedaço de palavra ou até pontuação. Use o tokenizer do próprio modelo de embeddings para garantir que a contagem esteja correta e alinhada com o que o modelo realmente processa.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

Metadados enriquecem os chunks e são o diferencial entre um RAG medíocre e um excelente

Quando você faz o chunking de um documento, cada chunk vem com metadados automáticos básicos – coordenadas dos elementos, número da página, subtítulos. Mas esses são metadados estruturais, não semânticos. Para ter metadados realmente úteis para o seu pipeline de RAG, você precisa construir sua própria lógica de extração. Esse trabalho de engenharia de dados é o que separa projetos que funcionam bem daqueles que falham.

O LangStruct, um projeto do Google, oferece uma abordagem elegante para esse problema: você descreve em linguagem natural o que quer extrair, fornece exemplos de qualidade mostrando a estrutura esperada, e o LLM faz o trabalho pesado de identificar e estruturar as informações. Por exemplo, para papers científicos, você define que quer extrair título, autores, afiliação e URL do repositório, dá exemplos realistas e o modelo retorna tudo organizado. O resultado pode ser salvo em JSONL e integrado ao pipeline de RAG como payload dos chunks no vector database, permitindo filtros e contextualização durante a busca.

> *Esse tipo de trabalho para a extração dos metadados e enriquecimento dos chunks é o que vai fazer a diferença na sua qualidade de retrieval ou na construção da sua base de conhecimento. Infelizmente, isso no mercado de trabalho hoje é muito negligenciado. Tem muito projeto falhando por falta de uma estrutura bem definida nessa etapa dos dados.*

Não subestime a etapa de metadados. Um chunk com o texto "a taxa de erro foi de 2.3%" é muito mais útil quando acompanhado de metadados como título, página e seção de origem. Esses

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

metadados permitem filtrar resultados durante a busca e fornecem contexto adicional ao LLM, melhorando significativamente a qualidade das respostas.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

O pipeline completo de ingestão une extração, chunking, embeddings e vector database em um fluxo coeso

Cada peça do RAG que vimos isoladamente – extração com Docling, chunking híbrido, geração de embeddings, armazenamento no Qdrant – precisa ser integrada em um pipeline coeso de ingestão de dados. Esse pipeline é o alicerce sobre o qual todo o sistema RAG se sustenta: se a ingestão for mal feita, nenhuma otimização no prompt ou no modelo vai compensar a baixa qualidade dos dados.

O pipeline final segue um fluxo claro: converter o documento com Docling, gerar chunks com o Hybrid Chunker (usando tokenizer alinhado ao modelo de embeddings e limite de 300 tokens), carregar metadados extraídos previamente, e inserir tudo no Qdrant. Um detalhe importante é o uso do FastEmbed, uma biblioteca integrada ao Qdrant que gera embeddings de forma otimizada usando ONNX Runtime, muito mais rápida que PyTorch ou TensorFlow na CPU. Com FastEmbed, você não precisa instanciar o modelo de embeddings separadamente – o Qdrant gera os vetores internamente durante a ingestão e durante as queries. O resultado é um sistema onde cada chunk carrega o texto original e os metadados no payload, e a busca retorna não apenas o conteúdo relevante mas também toda a rastreabilidade do documento de origem.

> *É importante usar o mesmo modelo no tokenizer e no embedding model porque eles precisam estar alinhados. O FastEmbed é uma solução leve e o Qdrant integra ele diretamente, então você pode gerar embeddings na hora da ingestão ou na hora da query sem precisar instanciar um modelo separado.*

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

O alinhamento entre tokenizer, modelo de embeddings e estratégia de chunking é o segredo técnico mais importante de um pipeline de RAG. Se você usa um modelo para contar tokens no chunking e outro para gerar embeddings, os tamanhos não vão bater e você terá chunks cortados ou informação perdida. Use sempre o mesmo modelo em todas as etapas.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

Resumo Final

A mensagem central do curso "Fundamentos de RAG" é clara: modelos de linguagem são poderosos geradores de texto, mas sozinhos não são confiáveis para responder perguntas baseadas em documentos específicos. A arquitetura RAG resolve isso ao combinar retrieval e generation – buscando primeiro os trechos relevantes em uma base de conhecimento vetorial e só então pedindo ao LLM para gerar a resposta fundamentada nesse contexto.

Mas o RAG não é apenas sobre conectar um retriever a um LLM. A qualidade do sistema depende de cada elo da cadeia: a extração dos dados com ferramentas como Docling, a estratégia de chunking que preserva o significado semântico, o enriquecimento com metadados que permite filtros e rastreabilidade, a escolha e alinhamento do modelo de embeddings, e o uso de vector databases como Qdrant para buscas rápidas e escaláveis. Negligenciar qualquer uma dessas etapas compromete todo o pipeline.

O curso demonstrou que é possível construir um sistema RAG funcional do zero, com poucas bibliotecas, e depois evoluir progressivamente para uma solução de produção. A base é simples – buscar, contextualizar e gerar – mas o diferencial está na engenharia dos dados. Projetos que investem tempo na qualidade da ingestão, no chunking inteligente e nos metadados conseguem respostas drasticamente melhores do que aqueles que pulam direto para o modelo.

Se você vai construir um sistema RAG, comece pelo pipeline de dados, não pelo modelo. Invista tempo na extração, no chunking e nos metadados antes de se preocupar com qual LLM usar. A qualidade da base de conhecimento é o fator que mais impacta a qualidade das respostas finais.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174

Sobre este resumo

Este resumo em formato Blinkist foi gerado a partir do curso **Fundamentos de RAG** de **Daniel Romero**.

O curso aborda os fundamentos da arquitetura Retrieval-Augmented Generation (RAG), desde a teoria dos embeddings e similaridade vetorial até a construção prática de pipelines de ingestão de dados e sistemas de busca semântica. O conteúdo cobre a implementação do zero em Python, o uso de vector databases como Qdrant, extração de dados com Docling, estratégias de chunking híbrido e enriquecimento de chunks com metadados.

Fonte: Curso online "Fundamentos de RAG" – Daniel Romero

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - IP: 187.59.186.174